Getting it running

One recipe, three situations

Florian Gärtner

Where we are today

	What this course is for
→	**Getting it running** ← you are here
	Try it out
	The terminal, and how to read what Claude does
	Lunch
	A working day with Claude Code
	What the model reads, predicts and weighs
	A model, a menu, and a loop
	It doesn’t remember. It’s handed everything again.
	Wrap, done at 17:00

Learning objectives

After this block you will be able to:

1. **Run** Claude Code: on your subscription, on the university's budget, or on both.

This block is a recipe

No theory. We get the tool running on your machine, and you leave with a file that starts it.

Which one are you?

Your situation	The launcher is
A subscription that has **never stopped** on you	your backup, for the day it does
A subscription that **stops mid-afternoon**	where you switch when it stops
No subscription	your way in

The five-hour wall

> ⚠ **Warning**
>
> A subscription gives you a fixed amount of use per five hours, as well as per week. When it runs out Claude simply stops, mid-task. Probably on a day you have a deadline and desperately need it.

So **everyone** builds the launcher today, subscription or not.

Step 1 — your university key

What you are getting

The HRZ offers its own AI service. It forwards your requests to the model providers and pays from a budget assigned to you, roughly **€50 a month**.

Access starts by email: you wrote to ki@uni-giessen.de, they sent an invitation link, and you set a password. With that you make a **personal key**: a long secret string beginning `sk-`.

No invitation yet? Then there is no key for you today. Follow along on your subscription or at a neighbour's screen, and send the request now.

> ⚠ **Pick the *personal* key**
>
> The portal also offers an "Agent" key. Despite the name, it is not for Claude Code and will not work here.

Get it now

We walk through the portal together. Three clicks, then **copy the key somewhere you can paste from**.

When you make the key, check that its models include the Claude ones (`anthropic/...`). A key without them is refused.

> ⚠ **Copy it cleanly**
>
> No quotation marks. No spaces before or after. This is the single most common reason the next step fails.

Step 2 — the launcher

Take the file

Your course folder is `github.com/F-Gaertner/course_ggs_2026_material`: **Code**, then **Download ZIP**, then unpack it. Its `launcher` folder holds two files: `claude-jlu.bat` for Windows and `claude-jlu.command` for the Mac. Copy yours into the folder `.local\bin` in your user folder (on the Mac, `.local/bin`). That is where the installer put Claude Code itself.

Windows | macOS

`claude-jlu.bat`. In File Explorer, paste `%USERPROFILE%\.local\bin` into the address bar and press Enter. Windows reads `%USERPROFILE%` as your own user folder. Copy the file there.

> **Why that folder**
>
> Your terminal looks for programs there. So the launcher starts from any terminal, including the one inside Positron or VS Code.

Put your key in it

Right-click the file and open it with a plain editor: Notepad on Windows, TextEdit on the Mac. A double-click would run it instead.

Find the one line that ends `sk-...` and replace `sk-...` with the key you just copied. Save.

Windows — `claude-jlu.bat` | macOS — `claude-jlu.command`

```
@echo off
REM Start Claude Code on the JLU (university) budget instead of a personal subscription.
REM Replace sk-... below with your own key, then save. Change nothing else.
setlocal
set ANTHROPIC_API_KEY=
set ANTHROPIC_BASE_URL=https://api.hrz.uni-giessen.de
set ANTHROPIC_AUTH_TOKEN=sk-...
claude %*
```

> ⚠ **One line, one change**
>
> Change nothing else. Every other line is needed as it is.

Run it

Open a terminal: PowerShell on Windows, Terminal on the Mac, or the one inside Positron or VS Code. Type the launcher's name and press Enter.

Windows | macOS

```
claude-jlu
```

Claude Code starts, and the first time it asks for a colour theme and whether you trust the folder it starts in: pick any theme, and say yes.

Before you type anything, switch to a smaller model: type `/model` and pick **Haiku**. Then ask Claude something. *"What can you do?"* will do.

> **Two ways in**
>
> If you have a subscription, you now have two ways to start the same tool: `claude` and `claude-jlu`. They bill to different accounts.

Step 3 — know who is paying

One command

As it starts, Claude Code says how it is billed: **API Usage Billing** when a key pays, your plan's name, such as **Claude Max**, on a subscription. Which key, and where it sends your requests, is in:

```
/status
```

On the university's budget it reads:

```
Auth token:          ANTHROPIC_AUTH_TOKEN
Anthropic base URL:  https://api.hrz.uni-giessen.de
```

> **The idea to remember**
>
> This is where "who is paying for this session" is written on your screen.

Everyone should be able to read their own before we move on.

When you crash into the five-hour wall

Your subscription stops mid-session and says so.

1. Type `/exit`
2. In the same terminal, type `claude-jlu --continue` (Mac: `claude-jlu.command --continue`)
3. Carry on where you left off

Nothing about your work changes. Not even the conversation. Only who pays for it.

Keep the budget alive

Two habits:

- **Pick a smaller model.** Type `/model` and pick **Haiku** for simple tasks or **Sonnet** for everyday work. The larger ones, Opus and Fable, cost many times as much. Check it each time you switch: Claude Code keeps the model you last picked
- **Type `/clear` when you change topic.** Everything you have said so far is re-sent with every new message, so a stale conversation costs money on every turn

When your subscription is back

Your subscription is already paid for, so switch back and save the university's budget for the next wall.

1. Type `/exit`
2. In the same terminal, type `claude --continue`
3. Type `/status` and check that it names your account and no university address

You do not need to log in again, because the launcher never touched your subscription login.

> ⚠ **`/model` remembers**
>
> A model you picked with `/model` stays picked for every new session, whichever way you start Claude Code. Pick your usual one again.

The one rule about this file

> ⚠ **Your key is a password**
>
> It sits in that file in plain text. Anyone who has it can spend your budget.

- Do not mail it
- Do not put it in a shared folder, Dropbox, or anything that syncs
- If you think someone has seen it, make a new key and paste that in instead

If it did not work

Put your hand up. In practice it is nearly always one of these:

What you see	Almost always
The key is refused	Quotation marks or a stray space around the key
`claude-jlu` is not recognized	The file is not in `.local\bin`, or saving renamed it, e.g. to `claude-jlu (1).bat`
Claude Code runs, but `/status` shows your personal account	You started `claude`, not the launcher
Claude Code asks you to choose a login method	You started `claude`, not the launcher. Or your file is an older copy, with the key in `ANTHROPIC_API_KEY`: take the current one
An error about model access	The key lacks the Claude models: make a new one with them

Which command, when

You want	Type
the university's budget	`claude-jlu` (Mac: `claude-jlu.command`)
your own subscription	`claude`
the last conversation back	either command, then `--continue`
an older conversation	either command, then `--resume`

Type it in a terminal in the folder your work lives in. Claude Code works in the folder you start it in. The terminal inside Positron or VS Code starts in your project's folder.

Whatever a slide shows after `claude`, you can type after `claude-jlu`.

What you have now

- You have Claude Code running
- You have a file that starts it on someone else's budget
- You can say, in one command, who is paying